

Taskdeck: In-Depth Analysis and Progress Report (Feb 2026)

1. Project Overview

Taskdeck is a **local-first kanban application** combining a .NET 8 backend with a Vue 3 + TypeScript frontend ¹. The system includes advanced automation features—proposal/approval flows, chat integrations, operations commands, archive management and worker services—while aiming to remain offline-friendly for solo developers and small teams ¹. Back-end components follow a **Clean Architecture** with Entity Framework Core (SQLite) for persistence and rely on request-correlation middleware, error mapping and other cross-cutting services ². The front-end uses Vue 3 with TypeScript, Vue Router and Pinia, offering modular routes for boards, archive, chat, ops CLI and log views ².

2. Implementation Snapshot

Backend

- **Layering** – The backend separates `Domain`, `Application`, `Infrastructure` and `API` projects, reflecting Clean Architecture best practices ².
- **Data storage** – EF Core + SQLite is the default; controllers are thin and cross-cutting services handle request correlation, error mapping and log timing ².
- **Authorization** – Most controllers are decorated with `[Authorize]` attributes; however some legacy controllers still allow caller-supplied actor IDs (through query/body) ². Claims-based identity is partially implemented but needs uniform adoption (see §5).
- **Automation** – Services like `AutomationProposalService`, `AutomationPlannerService`, `AutomationExecutorService` and `AutomationActorService` orchestrate proposal/planning/execution flows ². There's also an `OpsCliService`, `ArchiveService`, `ChatService` and `WorkerHeartbeatService` ².

Frontend

- **Vue 3 and TypeScript** – The UI implements routes for boards, proposals, operations CLI, archive, chat and logs; uses Pinia for state and custom composables for correlation IDs and error handling ².
- **Cross-cutting UI** – Includes correlation ID injection, keyboard shortcuts (work in progress) and typeahead selectors ².
- **Local-first stance** – The app still reads and writes to local SQLite; cloud hosting remains optional.

3. Current Phase Progress (Feb 17 2026)

The project plan is structured into four cycles. According to the `STATUS.md` snapshot, **Phases 1–3 are complete** and **Phase 4 is approximately 90 % complete** ³. Significant accomplishments include:

- **Auth infrastructure & baseline tests:** setup of JWT authentication and role/claim infrastructure; baseline integration tests and API error harness helpers ³.
- **Claims-first controllers:** board controllers now use claims-based identity; integration tests verify correct HTTP semantics ³.
- **Export/import flows:** board import/export flows with CSV/file integration and packaging infrastructure ³.
- **Automation workflows:** implementation of the automation proposal → plan → execute → chat/ops/log flows, including workers/heartbeat and queue management ³.
- **Archives and chat:** archive lifecycle (creating, listing, recovering boards) and chat services integrated ³.
- **Ops/logging:** operations CLI endpoints, logs UI and correlation ID middleware for tracing requests ³.
- **Maintainability refactors:** cross-layer cleanup, DRYing patterns and docs/CI governance (workflow concurrency, caching, docs cross-linking) ³.
- **Testing progress:** a comprehensive test suite of **733 tests** (477 backend, 245 frontend unit, 11 E2E) runs in CI across Windows and Linux ⁴.

Remaining items to finish Phase 4 include:

- Uniform **security/identity convergence:** retrofit remaining controllers to enforce `[Authorize]`, remove query/body actor IDs, and unify error semantics ⁵.
- **Cross-user existence policy:** fully adopt the 404/403 rules to hide cross-tenant resources ⁵.
- **LLM provider production path:** replace the mocked AI with a configurable environment-safe provider and add more planner/executor coverage ⁵.
- **Chat→project bootstrap:** allow AI chat to create boards or tasks from free-form conversation ⁵.
- **Database-level export/import:** general export/import across local DBs (currently only board-level export/import implemented) ⁵.
- **UX/Operator hardening:** unify keyboard accessibility, drag/edit guardrails, command palette enhancements and improved discoverability ⁵.

4. Issues and Backlog Analysis

Issue titles follow a structured prefix scheme (OPS, SEC, UX, PACK, AUTO, DEBT, etc.). Highlights by category:

4.1 Security (SEC)

- **SEC-01 (legacy controllers):** enforce auth on all remaining controllers; ensure protected endpoints require `[Authorize]` and update integration tests ⁶.
- **SEC-02:** migrate identity usage to claim-based actor, removing `userId` parameters; unify error semantics ⁷.
- **SEC-03:** add an authorization regression matrix covering unauthenticated, forbidden and cross-user access scenarios ⁸.

- **SEC-04:** standardize error contract with consistent error codes and messages ⁹ .

These issues align with the Phase 4 remaining tasks and Horizon A plan to converge on claims-based identity.

4.2 User Experience (UX)

UX issues emphasise usability and accessibility: - **UX-01:** archive lifecycle coherence across API and UI; ensure board restore flows match user expectations ¹⁰ .

- **UX-02:** drag/edit interaction safety guardrails and keyboard navigation in the command palette ¹⁰ .
- **UX-03:** activity selector discoverability; provide typeahead and keyboard selection across the app ¹⁰ .
- **UX-04/05:** shared input-assist scaffolding and escape behaviour contract (define how ESC key backs out of modals, boards, etc.) ¹⁰ .

4.3 Starter Packs / Prepackaged States (PACK)

Issues under **PACK** revolve around creating reusable board templates and deterministic QA fixtures: -

- **PACK-01:** define a starter pack manifest schema and RFC ¹¹ .
- **PACK-02:** implement backend apply endpoint with dry-run and conflict reporting ¹¹ .
- **PACK-03:** build a frontend catalog UI for browsing and applying packs ¹¹ .
- **PACK-04:** ship first-party starter packs (common labels, columns, board blueprints) ¹¹ .
- **PACK-05:** create deterministic QA/E2E fixture packs for consistent test states ¹¹ .

4.4 Automation (AUTO)

- **AUTO-01:** implement a real LLM provider abstraction; currently tests use mocks and need a production path ¹² .
- **AUTO-02:** harden planner/executor semantics; handle invalid or malicious proposals safely ¹² .

4.5 Technical Debt (DEBT)

- **DEBT-01:** reduce CS8618 nullable warnings and move to non-nullable reference types ¹³ .
- **DEBT-02:** improve log query scalability (queries are heavy; need efficiency improvements) ¹³ .
- **DEBT-03:** decide the scope and timeline for full database export/import implementation ¹³ .

4.6 Operations (OPS)

OPS issues focus on governance and backlog seeding: create GitHub project views, label taxonomy, PR templates, baseline verification run and doc governance ¹⁴ .

5. Pull Requests Overview

Major PRs provide context on completed work:

- **PR #25** – “Execute CODEX plan units 0–6” implemented early plan items: baseline infrastructure, claims-based boards controller, correlation ID middleware, CI pipeline updates and initial export/import flow. It unified error handling and added integration tests.
- **PR #15** – “Consolidate advanced services hardening, tests, and auth rollout design” introduced the automation stack (proposal/planner/executor), chat & ops CLI controllers, archive service, and worker heartbeats. It also integrated JWT auth and board access checks, though **[Authorize]** enforcement remained partially incomplete.

- **PR #65** – “Security: enforce auth on legacy controllers and expand 401 integration coverage” closed SEC-01 by adding `[Authorize]` to controllers that still allowed anonymous access and updated integration tests accordingly.
 - **PR #64/#61** – documentation clean-ups: removed redundant references, curated docs and cross-linked masterplan and status docs.
 - **PR #32/#28** – codified cross-user existence policy (404 vs 403) and added maintainability labels for automation and tech debt tasks.
- Collectively, these PRs show a consistent pattern of delivering plan units, implementing new services, closing security holes and improving documentation.

6. Roadmap and Horizon Planning

The **Implementation Masterplan** divides future work into three horizons ¹⁵ ¹⁶ ¹⁷ :

Horizon A (Weeks 1–2) – Security & Identity Convergence ¹⁵

1. Enforce `[Authorize]` on all controllers; remove query/body actor patterns; unify status codes and error contracts.
2. Expand unauthorized/forbidden integration tests; adopt cross-user existence policy.
3. Retrofit legacy services to use claims-based identity; implement new auth helpers and update docs accordingly.
4. Determine LLM provider selection (OpenAI, Azure or self-hosted).
5. Plan baseline export/import improvements.

Horizon B (Weeks 3–6) – Automation & Provider Hardening ¹⁶

1. Build production-ready LLM provider path with environment-safe keys and concurrency controls.
2. Expand planner/executor coverage to handle more operations and invalid states.
3. Improve archive/automation coherence (recover archived boards with proposals intact).
4. Introduce chat-to-project bootstrapping: allow AI chat to create boards & tasks.
5. Continue UI/UX improvements.

Horizon C (Weeks 7–12) – UX & Operability ¹⁷

1. Complete keyboard navigation & command palette with typeahead selection.
2. Enhance activity discovery and input assistance across UI.
3. Harden drag/edit interactions; add escape behaviour contract and help affordance.
4. Add instrumentation & operator UX (log filtering, progress indicators).
5. Finalize prepackaged starter packs and LLM sandbox features.

Active Backlog

Beyond horizon planning, the **active backlog** lists granular tasks such as finishing claims-first retrofit across controllers, adding regression tests, aligning archive lifecycle, implementing UX improvements, shipping starter packs and fixture packs, expanding the automation provider strategy and planning export/import implementation ¹⁸ .

7. Testing & CI

Taskdeck has an unusually rigorous test setup:

- **Backend tests:** Domain (148), Application (152), API integration (96), CLI integration (81). These use xUnit and run under Release configuration to avoid file locking issues ¹⁹ .
- **Frontend tests:** 245 unit tests (Vitest + Vue Test Utils) and typecheck/build tasks ²⁰ .
- **End-to-End (E2E):** 11 Playwright tests verifying critical user journeys such as board creation, moving cards, and automation flows ²¹ .
- **CI gates:** Six GitHub Actions workflows enforce docs governance, architecture boundaries, backend unit tests, API integration tests, frontend unit tests, and E2E smoke tests ²² . Each gate runs on Windows and Linux and caches dependencies to speed up runs ²² .

Coverage Map & Manual Testing

The `TESTING_GUIDE.md` includes a coverage map: domain invariants, application services, HTTP contracts, CLI behaviours, architecture boundaries, frontend units and E2E journeys are all covered ²³ . There is also a `MANUAL_TEST_CHECKLIST.md` for manual verification of complex workflows and offline/local-first scenarios ²⁴ .

8. Known Gaps & Risks

`STATUS.md` lists several risks ²⁵ :

1. **Security and identity:** legacy controllers still depend on caller-supplied user IDs; mixing claim & query identity can cause privilege escalation and inconsistent 401/403/404 semantics ²⁵ .
2. **Cross-user existence policy:** inconsistent adoption of the 403/404 policy may leak resource existence across tenants ²⁵ .
3. **LLM provider & planner:** current AI provider is a mock; switching to production may introduce cost/security/performance issues. Planner coverage is narrow and may not handle all board operations safely ²⁵ .
4. **Export/import at DB level:** not yet implemented; potential risk of data loss if not designed carefully ²⁵ .
5. **Log query performance:** heavy queries may degrade server responsiveness; needs optimization ²⁵ .
6. **Nullability warnings:** CS8618 warnings remain; may hide potential null reference bugs ²⁵ .
7. **Node version mismatch:** dev vs runtime Node versions may cause subtle build/test issues ²⁵ .
8. **UX gaps:** unresolved tasks like archive coherence, command palette keyboard navigation, activity discoverability, ops form ergonomics, drag/edit conflicts and escape contract ²⁵ .

9. Considerations & Recommendations

9.1 Security & Identity

- **Prioritise claims-based identity completion:** finish the retrofit across controllers by following Horizon A tasks. Add global auth filters to ensure all endpoints require authentication unless explicitly allowed. Maintain a strict 401/403/404 policy across all resources ⁶ .
- **Standardise error contracts:** adopt a consistent error response shape (e.g. `{ errorCode , message }`) and ensure tests verify them ⁹ .

- **Authorization matrix:** expand integration tests to cover cross-board and cross-user attempts. The risk register emphasises cross-user existence; verifying 404 responses for unauthorized resources will reduce data leak risk ²⁵ .

9.2 Automation & LLM Provider

- **LLM provider selection:** decide between using a hosted provider (OpenAI, Azure) vs. self-hosting. A hosted provider reduces maintenance but requires careful handling of API keys and cost monitoring; self-hosting may be complex for a solo dev.
- **Validation & guardrails:** harden the automation planner and executor to reject invalid or dangerous proposals (e.g. deleting all boards). Consider using deterministic state machines or domain policies to restrict AI-suggested changes.
- **Observability:** instrument automation flows with structured logs and metrics to detect when AI operations fail or behave unexpectedly.

9.3 Prepackaged States & Starter Packs

- **Manifest RFC:** finalize the package manifest schema and implement the apply endpoint with dry-run and conflict detection ²⁶ .
- **Starter packs:** deliver the first set (labels, columns, board templates) to accelerate onboarding and serve as deterministic E2E fixtures ²⁶ .
- **Package catalogue:** build a UI for browsing/applying packs; this will also support QA by providing reproducible test states.

9.4 UX & Operator Experience

- **Keyboard navigation & command palette:** finish the command palette with arrow/enter selection; implement ESC behaviour contract across modals and forms ¹⁰ .
- **Drag/edit guardrails:** ensure dragging cards does not trigger accidental edits; unify pointer vs keyboard interactions ¹⁰ .
- **Activity discoverability:** improve selector UIs with typeahead and inline search; unify across boards, tasks and automation triggers ¹⁰ .
- **Operator UI:** refine ops CLI and logs UI with filtering, search and progress indicators. This is part of Horizon C's emphasis on operability ¹⁷ .

9.5 Testing & CI

- **Expand E2E coverage:** while 11 E2E tests cover main flows, additional tests should cover new features (archive restore, starter packs, chat-to-project). Consider integrating cross-browser testing as the user base grows.
- **Maintain test data determinism:** use starter pack fixtures for E2E tests to ensure consistent state.
- **Coverage enforcement:** maintain current high coverage and review coverage drop warnings in CI ²² .
- **Automated performance tests:** as log queries and automation tasks may impact performance, add performance regression tests or benchmarks.

9.6 Future Scalability & Deployment

Although Taskdeck is local-first, future adoption may require scaling:

- **Database:** plan migration from SQLite to a more scalable DB (Postgres or SQL Server) when multi-team usage increases; ensure connection pooling and performance are tuned accordingly.
- **WebSockets/SignalR:** implement real-time updates for boards using SignalR to improve collaborative experience; this avoids heavy polling and scales better to many concurrent users.
- **Deployment:** containerize the application (Docker) and set up CI/CD pipelines for building and deploying to cloud infrastructure. Use Nginx or a cloud load balancer for TLS termination and scaling.
- **Caching & Rate limiting:** introduce distributed caching (e.g. Redis) for hot queries and implement API rate limits to protect against abuse.
- **Observability:** adopt structured logging (Serilog) and OpenTelemetry metrics/traces; deploy a monitoring stack (Grafana + Prometheus) to catch performance regressions.

9.7 Governance & Documentation

- **Docs governance cadence:** maintain weekly updates to `STATUS.md` and `IMPLEMENTATION_MASTERPLAN.md` as recommended ²⁷. The docs should remain the source of truth; update them whenever behaviour changes.
- **Backlog hygiene:** convert Horizon tasks and active backlog items into GitHub issues with labels; use project boards to manage WIP limits and enforce single major branch at a time (per recommendations in your `Next Work Checklist`).
- **Manual test scripts:** keep the `MANUAL_TEST_CHECKLIST.md` aligned with new features; manual tests remain important for local-first scenarios and for verifying offline behaviour.

10. Conclusion

Taskdeck has matured from a concept into a robust, feature-rich local-first kanban system with an impressive test harness and clear planning. Its current priorities are **security/identity convergence**, **automation hardening**, **starter pack infrastructure**, and **UX/operator polish**. The combination of Clean Architecture, comprehensive testing, rigorous documentation and structured planning positions the project well for growth. Completing the remaining security retrofits, implementing a production AI provider, shipping starter packs and refining user experience will bring Taskdeck from near-MVP to a professional, scalable product.

1 2 3 4 5 25 STATUS.md

<https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/STATUS.md>

6 SEC-01: Enforce auth on legacy controllers

<https://github.com/Chris0Jeky/Taskdeck/issues/58>

7 SEC-02: Claims-first identity (remove actor query/body IDs)

<https://github.com/Chris0Jeky/Taskdeck/issues/33>

8 SEC-03: Authz regression matrix tests

<https://github.com/Chris0Jeky/Taskdeck/issues/34>

9 SEC-04: Standardize API error contract assertions

<https://github.com/Chris0Jeky/Taskdeck/issues/44>

10 UX-02: Drag/edit interaction safety guardrails

<https://github.com/Chris0Jeky/Taskdeck/issues/45>

11 PACK-05: Deterministic QA/E2E fixture packs from manifests

<https://github.com/Chris0Jeky/Taskdeck/issues/51>

12 AUTO-02: Planner/executor hardening (coverage + failure semantics + audit quality)

<https://github.com/Chris0Jeky/Taskdeck/issues/40>

13 DEBT-02: Log query scalability pass

<https://github.com/Chris0Jeky/Taskdeck/issues/53>

14 OPS-06: Backlog seeding policy for docs -> issues (weekly cadence)

<https://github.com/Chris0Jeky/Taskdeck/issues/41>

15 16 17 18 26 27 IMPLEMENTATION_MASTERPLAN.md

https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/IMPLEMENTATION_MASTERPLAN.md

19 20 21 22 23 24 TESTING_GUIDE.md

https://github.com/Chris0Jeky/Taskdeck/blob/main/docs/TESTING_GUIDE.md